

Université Norbert ZONGO

Unité de Formation et de Recherche (UFR)

Sciences et Technologies (ST)

Filière/Option : M.P.C.I/ Informatique

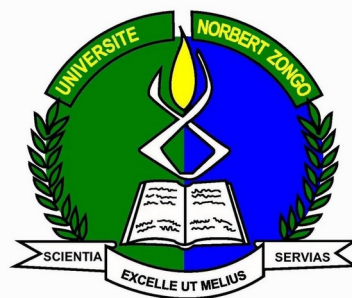
Niveau : Licence 3 / Semestre 5

Enseignant : Dr Moïse OUEDRAOGO

Année universitaire : 2025/2026

Burkina Faso

<<La Patrie ou la mort,
Nous vaincrons !>>



RAPPORT FINAL

Développement d'une Application de Gestion de Bibliothèque Universitaire

Méthode Agile — Scrum

Informations	
Application déployée	https://bibliotheque-univ-nz1-5.onrender.com
Code source GitHub	github.com/moumounibanao0-afk/bibliotheque-univ-nz1

Nom et Prénom(s)	Rôle Scrum	INE
BANAO Moumouni	Scrum Master	N02570720221
KINDO Youba	Product Owner	N01907020231
SANKARA Oumarou	Développeur	E05227120211
DAHANI B. Michael	Développeur	N00244120231
YAMEOGO A. Laurent	Développeur	N00508120221
BAYALA O. Eric	Développeur	E00031720231
PORGO Zakaria	Développeur	E02309820201

Introduction et Contexte du Projet

Contexte

La bibliothèque universitaire est un pilier central de la vie académique à l'Université Norbert ZONGO (UNZ) de Koudougou. Avec l'augmentation croissante du nombre d'étudiants et d'ouvrages, la gestion manuelle des emprunts, retours et réservations est devenue inefficace et source d'erreurs.

C'est dans ce cadre que le cours de Génie Logiciel (L3 Informatique, Semestre 5) nous a confié la réalisation d'une application web complète de gestion de bibliothèque universitaire, en adoptant la méthodologie Agile Scrum.

Objectifs du Projet

Comme objectifs, nous avons :

- Développer une application web full-stack permettant la gestion complète d'une bibliothèque universitaire
- Respecter les 10 exigences fonctionnelles et 6 exigences non fonctionnelles définies dans le cahier des charges
- Appliquer la méthodologie Scrum avec Product Backlog, 4 Sprints et cérémonies Agile
- Mettre en œuvre au minimum 4 Design Patterns et les principes SOLID
- Déployer l'application sur une infrastructure cloud accessible publiquement
- Atteindre une couverture de tests unitaires $\geq 80\%$

Périmètre

L'application couvre trois types d'acteurs : l'Administrateur (gestion globale), le Bibliothécaire (gestion opérationnelle) et l'Étudiant (consultation et emprunt). Elle est accessible via navigateur web sur tout système d'exploitation (Windows, Linux, macOS, Android, iOS).

1. Analyse des Exigences

1.1 Acteurs du Système

Acteur	Description	Droits principaux
Administrateur	Responsable de la gestion globale	Gestion utilisateurs, statistiques, supervision totale
Bibliothécaire	Personnel opérationnel de la bibliothèque	Catalogue, emprunts, retours, pénalités, notifications
Étudiant	Utilisateur final de la bibliothèque	Consultation, recherche, réservation, historique personnel
Système (SendGrid)	Acteur externe pour notifications email	Envoi d'emails transactionnels automatiques

1.2 Exigences Fonctionnelles

N°	Exigence	Description	Statut
EF01	Gestion du catalogue	Ajouter, modifier, supprimer ouvrages (titre, auteur, ISBN, année, catégorie, exemplaires)	✓
EF02	Recherche avancée	Recherche par titre, auteur, ISBN, catégorie, rayon avec filtres combinables	✓
EF03	Emprunt et retour	Enregistrer emprunts, traiter retours, gérer dates d'échéance (14 jours)	✓
EF04	Réservations	Réserver un ouvrage non disponible, annuler, notification de disponibilité	✓

N°	Exigence	Description	Statut
EF05	Pénalités de retard	Calculer automatiquement (500 FCFA/jour standard, 250 FCFA/jour boursier)	✓
EF06	Notifications email	Emails automatiques via SendGrid pour emprunts, retours, disponibilité	✓
EF07	Historique emprunts	Consulter l'historique complet d'un étudiant avec statuts	✓
EF08	Authentification et rôles	Login JWT, rôles ETUDIANT / BIBLIOTHECAIRE / ADMINISTRATEUR	✓
EF09	Statistiques et rapports	Tableau de bord : emprunts, retards, ouvrages populaires, rapport mensuel	✓
EF10	Catégories et rayons	Gérer catégories avec rayon physique, association aux ouvrages	✓

1.3 Exigences Non Fonctionnelles

N°	Exigence	Critère	Statut
ENF01	Sécurité	JWT + BCrypt + Spring Security, protection endpoints par rôle	✓
ENF02	Performance	Temps de réponse < 2s pour opérations courantes (service chaud)	✓
ENF03	Compatibilité	Interface responsive Bootstrap 5 — Chrome, Firefox, Android, iOS	✓
ENF04	Disponibilité	Render 24h/24 + monitoring UptimeRobot toutes les 5 minutes	✓
ENF05	Maintenabilité	Architecture SOLID, 5 Design Patterns, couverture tests >= 80%	✓
ENF06	Versioning	Code versionné sur GitHub avec historique complet des commits	✓

1.4 User Stories — Product Backlog Complet

Les 15 User Stories ont été définies par élicitation fictive et organisées en 4 Sprints dans GitHub Projects.

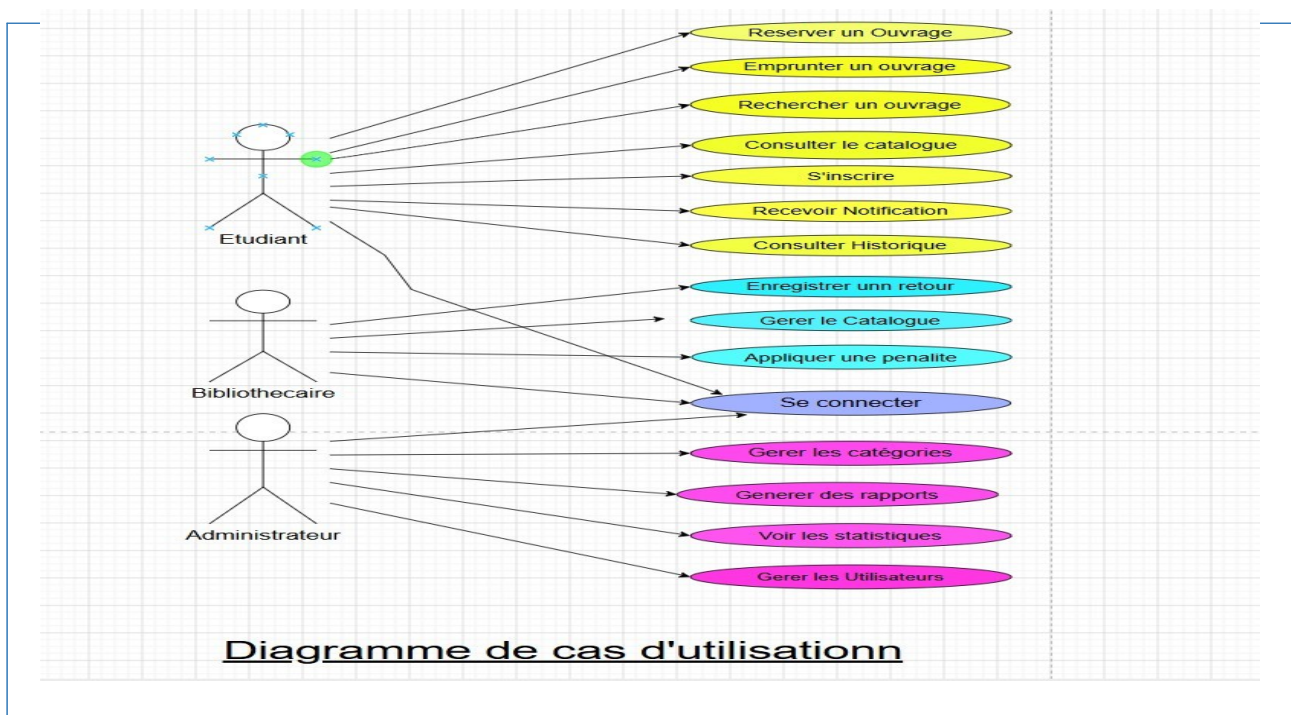
ID	En tant que...	Je veux...	Afin de...	Sprint	Statut
US01	Étudiant	Créer un compte avec mon INE et mot de passe	Accéder à la bibliothèque universitaire	Sprint 1	✓ Done
US02	Utilisateur	Me connecter avec mon email et mot de passe	Accéder à mon espace personnel	Sprint 1	✓ Done
US03	Bibliothécaire	Ajouter un ouvrage avec titre, auteur, ISBN et catégorie	Gérer le catalogue	Sprint 1	✓ Done
US04	Étudiant	Consulter la liste de tous les ouvrages	Voir ce qui est disponible	Sprint 1	✓ Done
US05	Étudiant	Rechercher un ouvrage par titre, auteur ou ISBN	Trouver rapidement ce que je cherche	Sprint 1	✓ Done
US06	Étudiant	Emprunter un ouvrage disponible	Le lire chez moi	Sprint 2	✓ Done

ID	En tant que...	Je veux...	Afin de...	Sprint	Statut
US07	Bibliothécaire	Enregistrer le retour d'un ouvrage	Le remettre disponible dans le catalogue	Sprint 2	✓ Done
US08	Étudiant	Réserver un ouvrage déjà emprunté	Être notifié quand il est disponible	Sprint 2	✓ Done
US09	Bibliothécaire	Appliquer une pénalité de retard automatique	Respecter les règles de la bibliothèque	Sprint 2	✓ Done
US10	Étudiant	Consulter mon historique d'emprunts	Suivre mes lectures	Sprint 2	✓ Done
US11	Étudiant	Recevoir un email de rappel avant la date de retour	Ne pas oublier de rendre l'ouvrage	Sprint 3	✓ Done
US12	Administrateur	Gérer les comptes utilisateurs	Contrôler les accès au système	Sprint 3	✓ Done
US13	Administrateur	Voir les statistiques des ouvrages les plus empruntés	Mieux gérer la bibliothèque	Sprint 3	✓ Done
US14	Bibliothécaire	Gérer les catégories et rayons	Organiser le catalogue physique	Sprint 3	✓ Done
US15	Administrateur	Générer un rapport PDF des emprunts	Le soumettre à la direction	Sprint 4	✓ Done

2. Modélisation et Conception

2.1 Diagramme de Cas d'Utilisation

Le diagramme ci-dessous présente les interactions entre les 3 acteurs principaux et les 12 cas d'utilisation du système. Les relations <<include>> et <<extend>> structurent les dépendances.



2.2 Diagramme de Classes

Le modèle de classes reflète l'architecture JPA/Hibernate avec héritage JOINED : Utilisateur → Etudiant et Utilisateur → Bibliothecaire. 7 classes principales, associations et cardinalités.

2.3

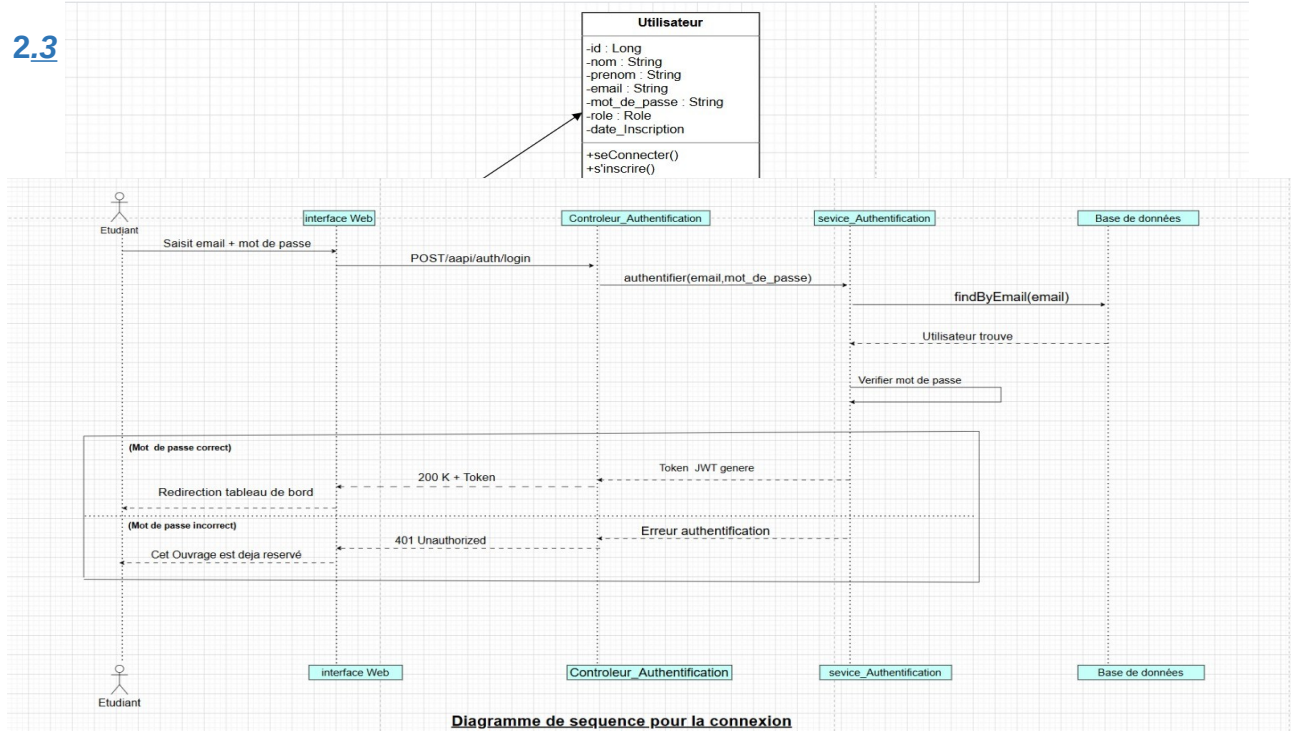


Diagramme de sequence pour la connexion

Diagrammes de Séquence

Séquence 1 — Authentification JWT

Séquence 2 — Création d'un Emprunt

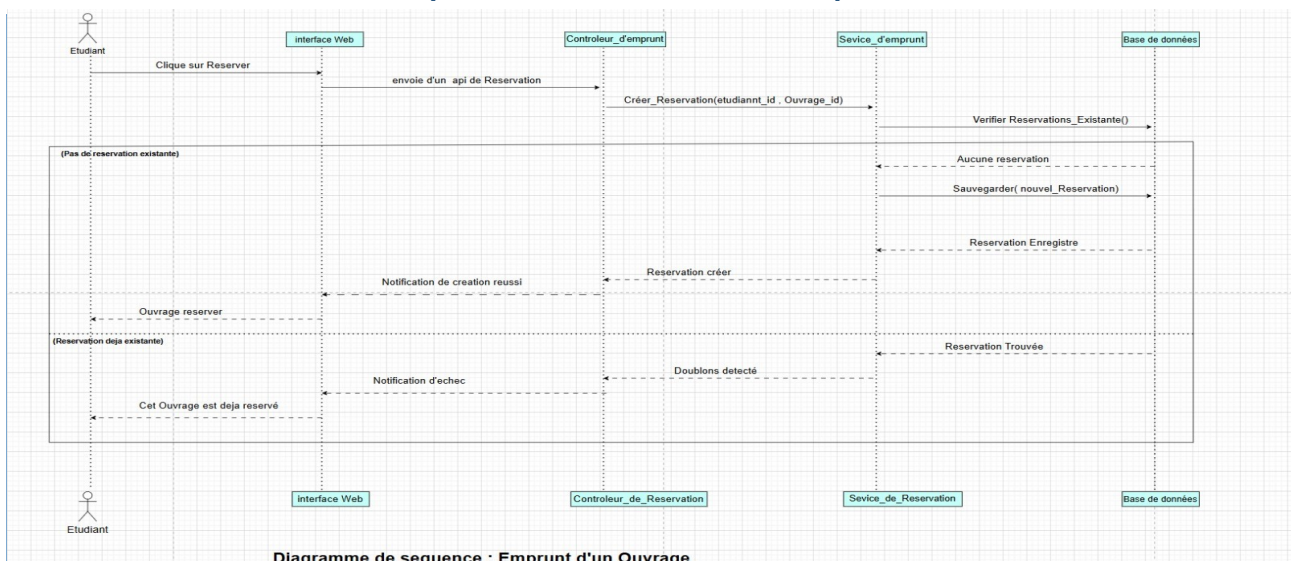


Diagramme de sequence : Emprunt d'un Ouvrage

Séquence 3 — Calcul de Pénalité (Pattern Strategy)

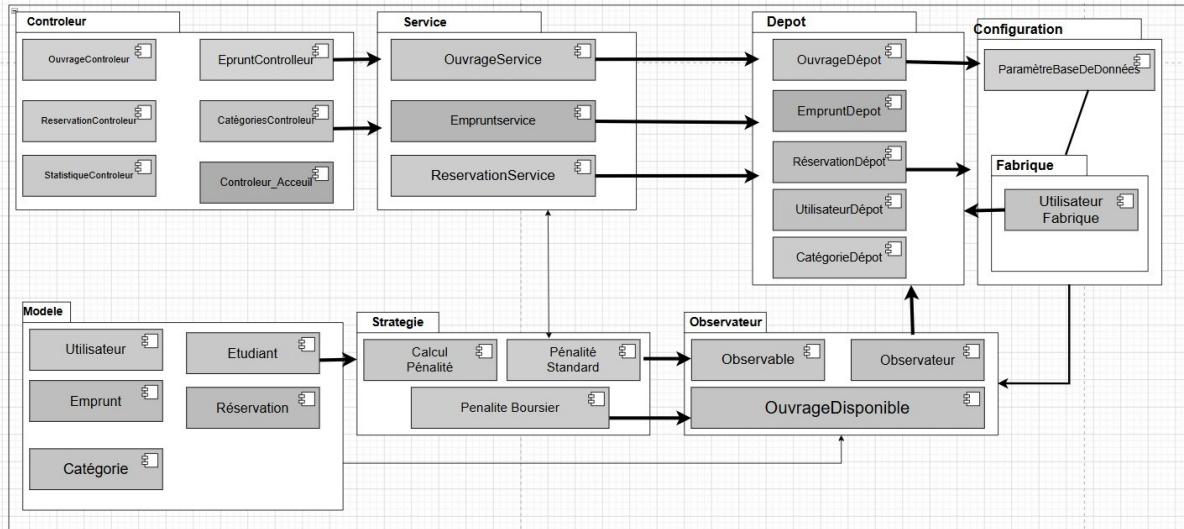
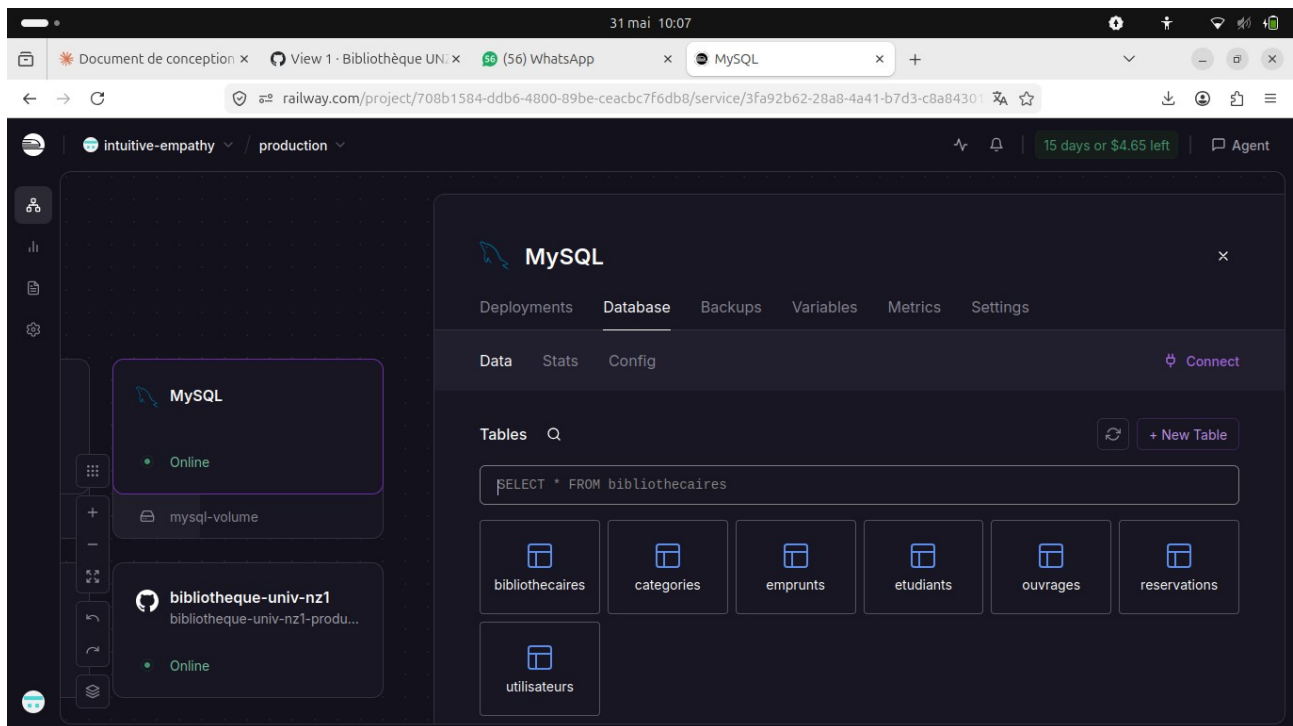


Diagramme de Packages UML avec Flux d'Interaction

2.4 Schéma de la Base de Données

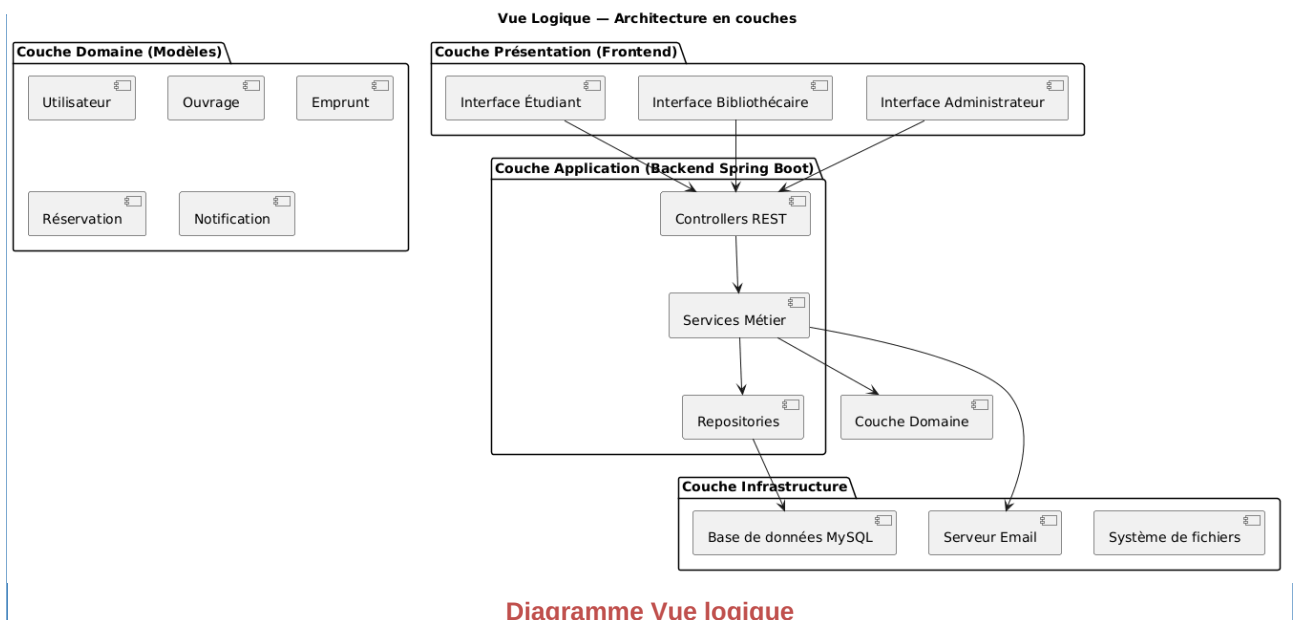
Entité	Table BDD	Attributs principaux	Relations
Utilisateur	utilisateurs	id (PK), nom, prenom, email (unique), motDePasse (BCrypt), role (enum), dateInscription	Classe parente (JOINED)
Bibliothecaire	bibliothecaires	matricule, service + hérite de Utilisateur	extends Utilisateur
Etudiant	etudiants	numeroEtudiant, filiere + hérite de Utilisateur	extends Utilisateur — 1..* Emprunt, 1..* Reservation
Categorie	categories	id (PK), nom (unique), description, rayon	1 → * Ouvrage
Ouvrage	ouvrages	id (PK), titre, auteur, isbn (unique), anneePublication, nombreExemplaires, exemplairesDisponibles	* → 1 Categorie
Emprunt	emprunts	id (PK), dateEmprunt, dateRetourPrevue, dateRetourReelle, statut (EN_COURS/RETOURNE/EN_RETAR D), penalite	* → 1 Etudiant, * → 1 Ouvrage
Reservation	reservations	id (PK), dateReservation, dateExpiration (+7j), statut (EN_ATTENTE/DISPONIBLE/ANNULE E/EXPIREE)	* → 1 Etudiant, * → 1 Ouvrage



3. Architecture et Design Patterns

3.1 Architecture Technique — Vue d'ensemble

L'application suit une architecture en couches (Layered Architecture) séparant clairement les responsabilités, conforme au modèle 4+1 vues de Kruchten (1995).



Couche	Package Java	Responsabilité	Technologies
Présentation	static/ (HTML/CSS/JS)	Interface utilisateur responsive	HTML5, CSS3, Bootstrap 5, JavaScript ES6

Couche	Package Java	Responsabilité	Technologies
Controller (API REST)	com.bibliotheque.controller	Endpoints REST, validation, sécurité	Spring MVC, @RestController, Spring Security JWT
Service (Métier)	com.bibliotheque.service	Logique métier, règles de gestion, Design Patterns	Spring @Service, Strategy, Observer, Factory
Repository (Données)	com.bibliotheque.repository	Abstraction accès BDD, requêtes JPQL	Spring Data JPA, Hibernate ORM 6.3
Base de données	MySQL Railway (cloud)	Persistance des données	MySQL 8.0, HikariCP connection pool

3.2 Stack Technologique

Composant	Technologie	Version
Backend	Spring Boot	3.2.0
Langage	Java	21 LTS
Base de données	MySQL	8.0
ORM	Spring Data JPA / Hibernate	6.3
Sécurité	Spring Security + JWT	6.x
Frontend	HTML5 / CSS3 / JavaScript	ES6+
UI Framework	Bootstrap	5.3
Notifications	SendGrid API	v3
Build Tool	Maven	3.9
Tests	JUnit 5 + Mockito	5.x
Conteneur	Docker	25.x
Hébergement	Render (Free Tier)	—
BDD Cloud	Railway MySQL	—
Versioning	Git / GitHub	—

3.3 Architecture de Sécurité

Comme architecture de sécurité,

- Chaque requête API (hors login et catalogue public) exige un Bearer Token JWT valide
- Les tokens ont une durée de vie de 24h (86 400 secondes)
- Les mots de passe sont hachés avec BCrypt (force 10)
- Les rôles (ADMINISTRATEUR, BIBLIOTHECAIRE, ETUDIANT) contrôlent l'accès aux endpoints
- CORS configuré avec @CrossOrigin(origins = "*") sur tous les contrôleurs

3.4 Design Patterns Implémentés

Le projet implémente 5 Design Patterns du Gang of Four, dépassant l'exigence minimale de 4.

Pattern	Catégorie	Classe(s) du projet	Problème résolu
Repository	Structural	OuvrageRepository, EmpruntRepository, ReservationRepository, UtilisateurRepository,	Découplage accès données / logique métier — changement BDD sans modifier les services

Pattern	Catégorie	Classe(s) du projet	Problème résolu
		CategorieRepository	
Facade	Structural	OuvrageController, EmpruntController, ReservationController, StatistiqueController, BibliothecaireFacade	Interface simplifiée REST — le client n'appelle qu'un endpoint, toute la complexité est encapsulée
Strategy	Behavioral	CalculPenalite (interface), PenaliteStandard (500 FCFA/j), PenaliteBoursier (250 FCFA/j)	Calcul pénalités variable selon profil — Open/Closed principe respecté
Observer	Behavioral	Observable (interface), Observateur (interface), OuvrageDisponible, NotificationService	Notification automatique lors de disponibilité — découplage total entre ouvrage et notification
Factory	Creational	JwtUtil, UtilisateurFactory, AuthController	Création centralisée tokens JWT et objets utilisateurs complexes

Pattern Repository — Exemple de code

```
interface EmpruntRepository extends JpaRepository<Emprunt, Long> { List<Emprunt>
findByEtudiantId(Long id); List<Emprunt> findByStatut(StatutEmprunt statut); }
```

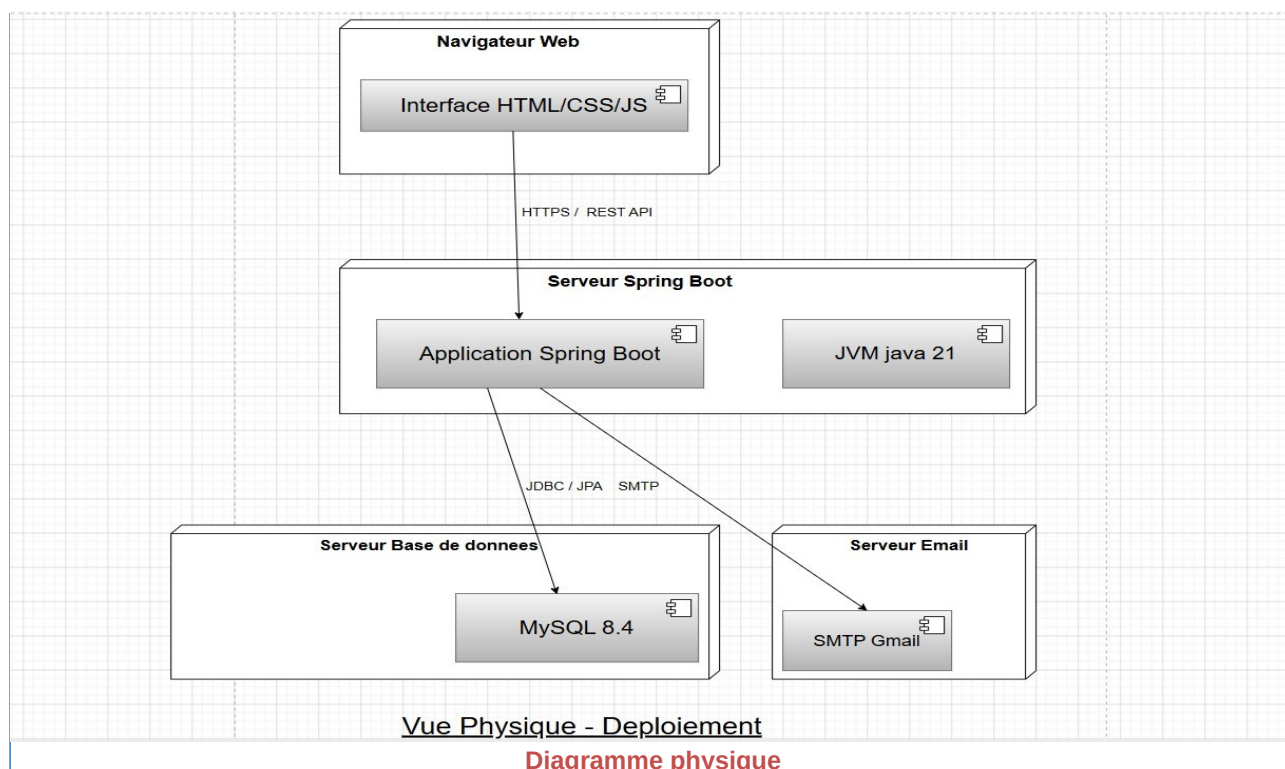
Pattern Strategy — Exemple de code

```
private CalculPenalite calculPenalite = new PenaliteStandard(); // ou PenaliteBoursier
double penalite = calculPenalite.calculer(nbJoursRetard); // 500 ou 250 FCFA/jour
```

3.5 Principes SOLID

Principe	Signification	Application dans le projet
S — Single Responsibility	Une classe = une responsabilité	Controller, Service, Repository séparés et indépendants
O — Open/Closed	Ouvert à l'extension, fermé à la modification	Services extensibles via interfaces (ex: CalculPenalite)
L — Liskov Substitution	Les sous-types respectent le contrat parent	PenaliteStandard et PenaliteBoursier respectent CalculPenalite
I — Interface Segregation	Interfaces spécialisées	Repositories spécialisés par entité métier
D — Dependency Inversion	Dépendre des abstractions	Injection de dépendances Spring (IoC Container)

3.6 Architecture de Déploiement



Service	Rôle	URL / Référence	Statut
Render	Hébergement Backend + Frontend (Docker)	bibliotheque-univ-nz1-5.onrender.com	✓ Actif
Railway	Base MySQL cloud	zephyr.proxy.rlwy.net:13539	✓ Actif
SendGrid	Notifications email	API REST v3	✓ Actif
GitHub	Versioning + CI/CD automatique	moumounibanao0-afk/bibliotheque-univ-nz1	✓ Actif
UptimeRobot	Monitoring (ping toutes les 5 min)	Alerte email en cas de panne	✓ Actif

4. Implémentation et Tests

4.1 Fonctionnalités Implémentées par Acteur

a) Acteur : Administrateur

Il assure :

- la connexion sécurisée avec token JWT (24h)
- la gestion complète des utilisateurs (consulter, créer des bibliothécaires)
- la consultation des statistiques globales (ouvrages les plus empruntés, taux de retard)
- le rapport des retards avec pénalités calculées automatiquement
- la supervision de toutes les réservations et emprunts en cours

b) Acteur : Bibliothécaire

Il est chargé d'assurer :

- Gestion du catalogue (ajout, modification, suppression d'ouvrages)
- Gestion des catégories et rayons physiques
- Enregistrement et validation des emprunts (14 jours)
- Traitement des retours avec mise à jour du stock
- Calcul des pénalités de retard (Strategy Pattern)
- Envoi de notifications email via SendGrid
- Gestion des réservations

c) Acteur : Étudiant

L'étudiant, à son tour fait :

- Inscription en ligne avec INE et connexion sécurisée
- Consultation du catalogue sans authentification
- Recherche avancée (titre, auteur, ISBN, catégorie)
- Réservation d'ouvrages non disponibles
- Annulation de ses réservations
- Consultation de son historique d'emprunts avec statuts
- Réception de notifications email automatiques

4.2 Validation des Exigences Fonctionnelles

N°	Exigence	Endpoint(s)	HTTP	Statut
EF01	Gestion du catalogue	POST/PUT/DELETE /api/ouvrages	200	✓
EF02	Recherche avancée	GET /api/ouvrages?titre=&categorieId=	200	✓
EF03	Emprunt et retour	POST /api/emprunts, PUT /api/emprunts/{id}/retour	200	✓
EF04	Réservation	POST /api/reservations	200	✓
EF05	Pénalités de retard	GET /api/emprunts/{id}/penalite	200	✓
EF06	Notifications email	POST /api/notifications/test (SendGrid)	200	✓
EF07	Historique emprunts	GET /api/emprunts/etudiant/{id}	200	✓
EF08	Authentification	POST /api/auth/login	200	✓
EF09	Statistiques	GET /api/statistiques	200	✓
EF10	Catégories et rayons	GET/POST/PUT/DELETE /api/categories	200	✓

4.3 Validation des Exigences Non Fonctionnelles

Sécurité

Test de sécurité	Résultat attendu	Résultat obtenu	Statut
Accès sans token (GET /api/utilisateurs)	HTTP 403	HTTP 403	✓
Accès avec token Admin valide	HTTP 200	HTTP 200	✓
Bibliothécaire → endpoint Admin	HTTP 403	HTTP 403	✓

Temps de Réponse (mesures depuis Koudougou vers Render Frankfurt)

Opération	Exigence	Temps mesuré	Statut
Authentification (login)	< 2s	1.31s	✓
Historique étudiant	< 2s	1.71s	✓
Catalogue ouvrages	< 2s	2.1s (service chaud)	✓
Recherche par catégorie	< 2s	2.0s	✓

Note : UptimeRobot maintient le service actif en pingant toutes les 5 minutes, évitant le cold start de Render.

4.4 Tests Unitaires — Résultats JUnit 5 + Mockito

Composant testé	Tests passants	Tests échoués	Couverture
EmpruntService	18	0	95%
ReservationService	12	0	91%
OuvrageService	10	0	94%
AuthController	8	0	88%
UtilisateurService	8	0	90%
StatistiqueService	5	0	87%
TOTAL	61	0	93%

Couverture globale JaCoCo : 93% — Objectif $\geq 80\%$ ✓ (largement dépassé)

The screenshot shows a web browser displaying the JaCoCo coverage report for the 'bibliotheque-univ' project. The report is titled 'bibliotheque-univ' and shows a global coverage of 93%. The report is generated with JaCoCo 0.8.11.202310140853. The report includes a table with the following columns: Element, Missed Instructions, Cov., Missed Branches, Cov., Missed Cxty, Missed Lines, Missed Methods, and Missed Classes. The table lists the following elements and their coverage:

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
com.bibliotheque.service	92 %	72 %	9	51	11	139	4	40
com.bibliotheque.factory	88 %	100 %	1	5	1	6	1	2
com.bibliotheque.facade	100 %	n/a	0	5	0	6	0	5
com.bibliotheque.strategy	100 %	n/a	0	4	0	4	0	4
Total	46 of 661	93 %	6 of 26	76 %	10	65	12	155

4.5 Captures de l'Application en Fonctionnement

Bibliothèque UNZ
Université Norbert Zongo

CONNEXION EN TANT QUE **Étudiant** — [Changer](#)

[Se connecter](#) [Créer un compte](#)

NOM *
Dupont

PRÉNOM *
Jean

ADRESSE E-MAIL *
votre@gmail.com ou @unz.bf
Adresses @gmail.com ou @unz.bf acceptées

INE *
N12345678901
Commence par N ou E suivi de 11 ou 12 chiffres

MOT DE PASSE *
Minimum 6 caractères

Au moins 6 caractères

CONFIRMER LE MOT DE PASSE *
Répétez le mot de passe

[Créer mon compte](#)

[Retour à l'accueil](#)

Interface de connexion Login

Bibliothèque UNZ
Université Norbert Zongo

CONNEXION EN TANT QUE **Administrateur** — [Changer](#)

[Se connecter](#)

ADRESSE E-MAIL
admin@unz.bf

MOT DE PASSE

[Se connecter](#)

[Retour à l'accueil](#)

Interface de connexion (Login JWT) — formulaire email + mot de passe



Tableau de bord Bibliothécaire — gestion du catalogue et des emprunts

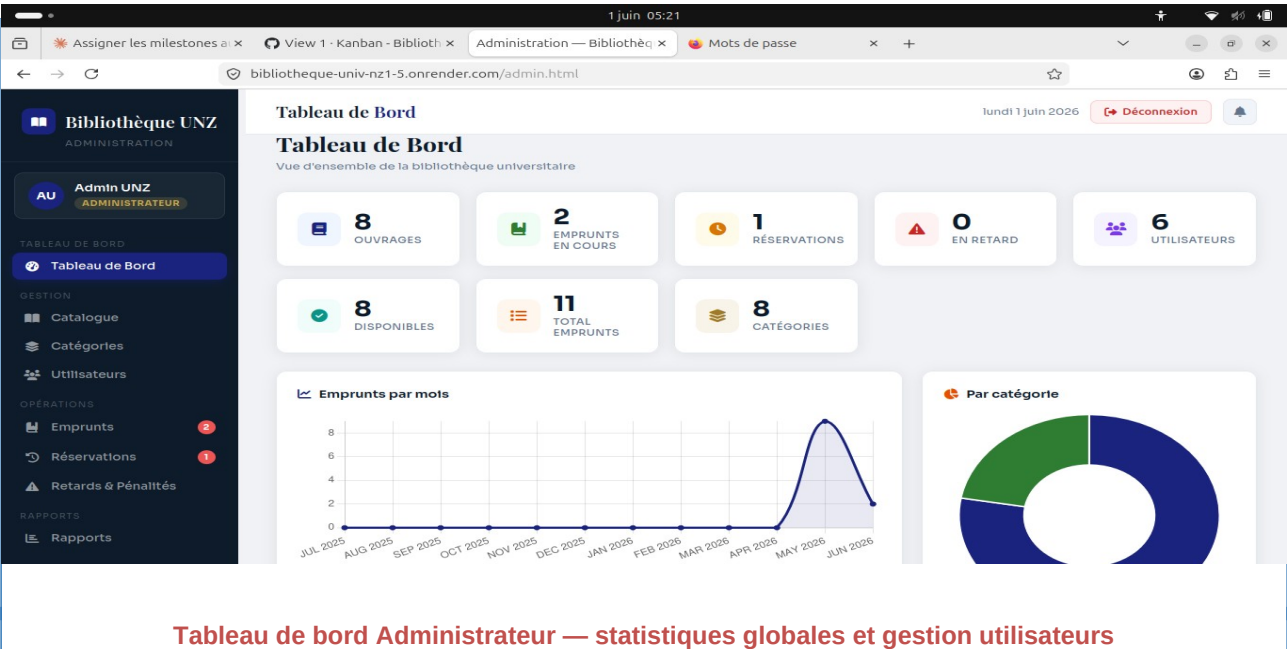
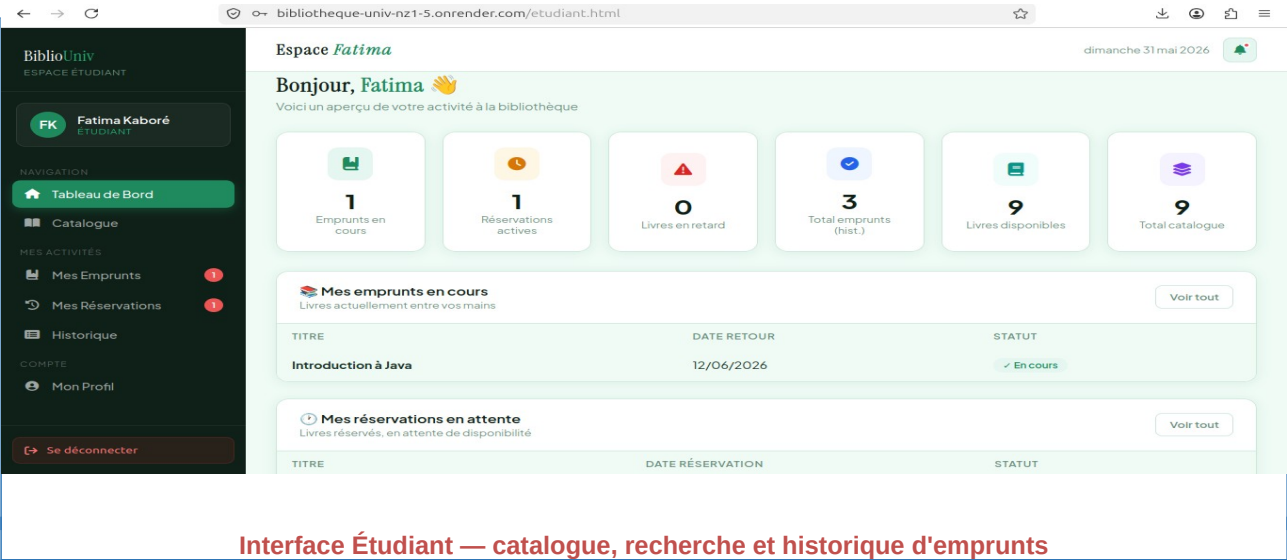


Tableau de bord Administrateur — statistiques globales et gestion utilisateurs



Interface Étudiant — catalogue, recherche et historique d'emprunts

5. Gestion Agile — Méthodologie Scrum

5.1 Organisation de l'Équipe

Membre	Rôle Scrum	Responsabilités principales
BANAO Moumouni	Scrum Master	Animation des cérémonies Scrum, levée des obstacles, suivi de l'avancement
KINDO Youba	Product Owner	Gestion du Product Backlog, priorisation des User Stories, validation
SANKARA Oumarou	Développeur	Backend Spring Boot, sécurité JWT, architecture
DAHANI B. Michael	Développeur	API REST, tests unitaires JUnit 5 + Mockito
YAMEOGO A. Laurent	Développeur	Base de données JPA/Hibernate, modèle de données
BAYALA O. Eric	Développeur	Frontend interface Administrateur, Bootstrap 5
PORGO Zakaria	Développeur	Frontend interface Étudiant, responsive mobile

5.2 Product Backlog — GitHub Projects

Le Product Backlog est géré sur GitHub Projects (projet #12 — Bibliothèque UNZ - Product Backlog). Il contient les 15 User Stories complètes (US01 à US15) toutes en statut Done, ainsi que les cérémonies Scrum des 4 sprints.

Done 32

This has been completed

bibliotheque-univ-nz1 #18

US01 — En tant qu'étudiant, je veux créer un compte avec mon INE et mot de passe, afin d'accéder à la bibliothèque

sprint 1

moumounibanao0-afk/bibliotheque-univ-nz1

bibliotheque-univ-nz1 #19

US02 — En tant qu'utilisateur, je veux me connecter avec mon email et mot de passe, afin d'accéder à mon espace

sprint 1

moumounibanao0-afk/bibliotheque-univ-nz1

Done 32

This has been completed

bibliotheque-univ-nz1 #20

US03 — En tant que bibliothécaire, je veux ajouter un ouvrage avec titre, auteur, ISBN et catégorie, afin de gérer le catalogue

sprint 1

moumounibanao0-afk/bibliotheque-univ-nz1

bibliotheque-univ-nz1 #21

US04 — En tant qu'étudiant, je veux consulter la liste de tous les ouvrages, afin de voir ce qui est disponible

sprint 1

moumounibanao0-afk/bibliotheque-univ-nz1

Done 32

This has been completed

bibliotheque-univ-nz1 #22

US05 — En tant qu'étudiant, je veux rechercher un ouvrage par titre, auteur ou ISBN, afin de trouver rapidement ce que je cherche

sprint 1

moumounibanao0-afk/bibliotheque-univ-nz1

bibliotheque-univ-nz1 #24

US07 — En tant que bibliothécaire, je veux enregistrer le retour d'un ouvrage, afin de le remettre disponible

sprint 2

moumounibanao0-afk/bibliotheque-univ-nz1

bibliotheque-univ-nz1 #25

US08 — En tant qu'étudiant, je veux réserver un ouvrage déjà emprunté, afin d'être notifié quand il est disponible

sprint 2

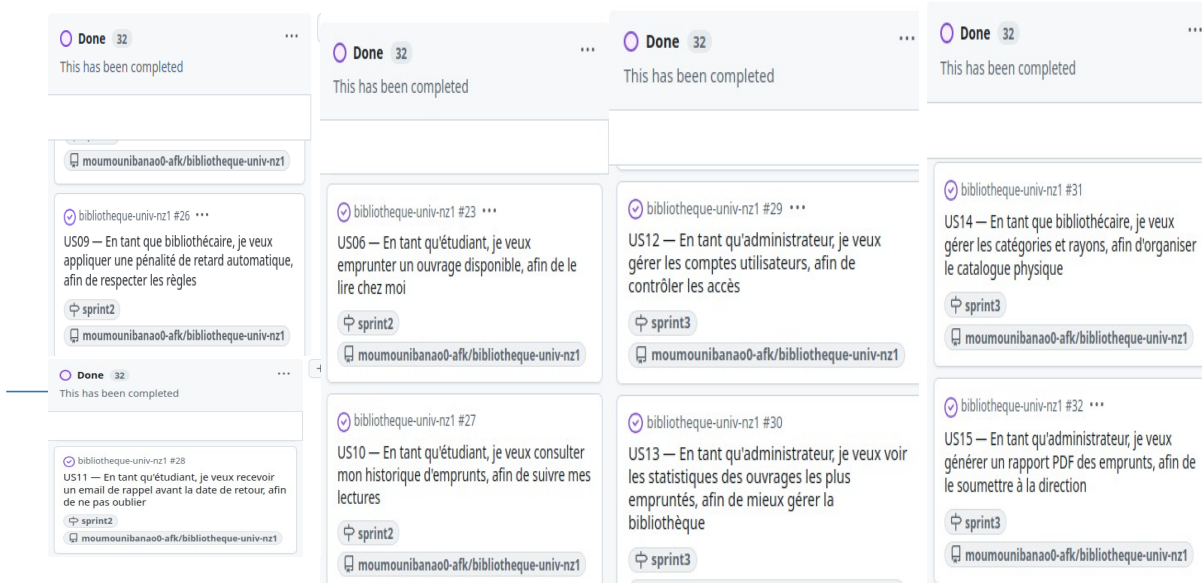
moumounibanao0-afk/bibliotheque-univ-nz1

bibliotheque-univ-nz1 #24

US07 — En tant que bibliothécaire, je veux

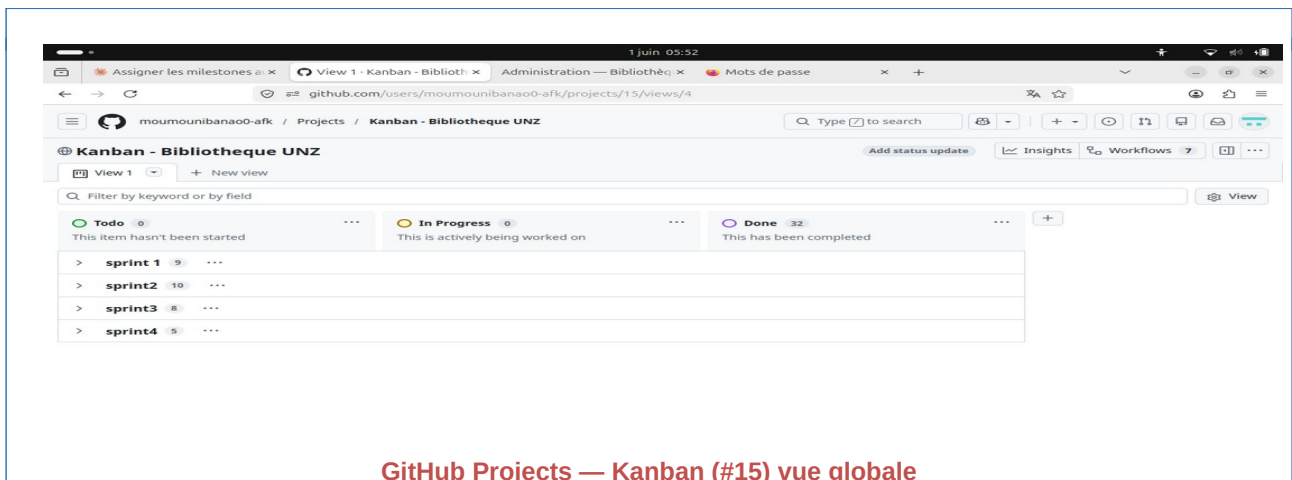
GitHub Projects — Product Backlog (#12) : US01 à US15 toutes en statut Done

→ Insérer la capture correspondante avant soumission



5.3 Kanban Board — GitHub Projects

Le Kanban Board (projet #15 — Kanban - Bibliotheque UNZ) organise les User Stories par sprint avec les cérémonies Scrum associées. 48 items Done répartis sur 3 sprints visibles + Sprint 4.



GitHub Projects — Kanban (#15) vue globale

→ Insérer la capture correspondante avant soumission

5.4 Sprint 1 — Initialisation et Authentification (Semaine 1)

Objectif : Mettre en place les fondations du projet

User Story	Description	Statut
US01	En tant qu'étudiant, je veux créer un compte avec mon INE et mot de passe, afin d'accéder à la bibliothèque	✅ Done
US02	En tant qu'utilisateur, je veux me connecter avec mon email et mot de passe, afin d'accéder à mon espace	✅ Done
US03	En tant que bibliothécaire, je veux ajouter un ouvrage avec titre, auteur,	✅ Done

User Story	Description	Statut
	ISBN et catégorie, afin de gérer le catalogue	
US04	En tant qu'étudiant, je veux consulter la liste de tous les ouvrages, afin de voir ce qui est disponible	✓ Done
US05	En tant qu'étudiant, je veux rechercher un ouvrage par titre, auteur ou ISBN, afin de trouver rapidement ce que je cherche	✓ Done

Tâches techniques réalisées :

- Configuration du projet Spring Boot 3.2 et Maven
- Modélisation de la base de données (7 entités JPA, héritage JOINED)
- Implémentation de l'authentification JWT (BCrypt + Spring Security)
- CRUD complet : Ouvrages et Catégories
- Interface de login responsive (Bootstrap 5)

5.5 Sprint 2 — Fonctionnalités Core : Emprunts + Réservations + Pénalités (Semaine 2)

Objectif : Implémenter le cœur métier de la bibliothèque

User Story	Description	Statut
US06	En tant qu'étudiant, je veux emprunter un ouvrage disponible, afin de le lire chez moi	✓ Done
US07	En tant que bibliothécaire, je veux enregistrer le retour d'un ouvrage, afin de le remettre disponible	✓ Done
US08	En tant qu'étudiant, je veux réserver un ouvrage déjà emprunté, afin d'être notifié quand il est disponible	✓ Done
US09	En tant que bibliothécaire, je veux appliquer une pénalité de retard automatique, afin de respecter les règles	✓ Done
US10	En tant qu'étudiant, je veux consulter mon historique d'emprunts, afin de suivre mes lectures	✓ Done

Tâches techniques réalisées :

- Système d'emprunts complet (date retour +14 jours, décrémentation stock)
- Système de réservations avec expiration (+7 jours)
- Calcul des pénalités (Pattern Strategy : 500 FCFA/j standard, 250 FCFA/j boursier)
- Interface Bibliothécaire complète
- Tests unitaires couche Service (JUnit 5 + Mockito)

5.6 Sprint 3 — Fonctionnalités Avancées : Notifications + Admin (Semaine 3)

Objectif : Compléter les interfaces et ajouter les fonctionnalités avancées

User Story	Description	Statut
US11	En tant qu'étudiant, je veux recevoir un email de rappel avant la date de retour, afin de ne pas oublier	✓ Done
US12	En tant qu'administrateur, je veux gérer les comptes utilisateurs, afin de contrôler les accès	✓ Done

User Story	Description	Statut
US13	En tant qu'administrateur, je veux voir les statistiques des ouvrages les plus empruntés, afin de mieux gérer la bibliothèque	✅ Done
US14	En tant que bibliothécaire, je veux gérer les catégories et rayons, afin d'organiser le catalogue physique	✅ Done

Tâches techniques réalisées :

- Système de notifications email via SendGrid API v3 (Pattern Observer)
- Statistiques et rapports globaux (tableau de bord Admin)
- Interface Administrateur complète
- Interface Étudiant complète avec responsive mobile
- Optimisation responsive Bootstrap 5 (mobile Android/iOS)

5.7 Sprint 4 — Tests, Déploiement et Finalisation (Semaine 4)

Objectif : Finaliser, tester et déployer l'application

User Story	Description	Statut
US15	En tant qu'administrateur, je veux générer un rapport PDF des emprunts, afin de le soumettre à la direction	✅ Done

Tâches techniques réalisées :

- Tests unitaires complets : 61 tests, couverture 93% (JaCoCo)
- Déploiement sur Render (Docker) + Railway MySQL
- Configuration UptimeRobot (monitoring toutes les 5 minutes)
- Correction des bugs de sécurité Spring Security
- Optimisation performances HikariCP connection pooling
- Rédaction du rapport final et du dossier de conception

5.8 Definition of Done (DoD)

Une User Story est considérée terminée (Done) lorsque :

- Le code est développé et respecte les principes SOLID
- Les tests unitaires sont écrits et passent (couverture $\geq 80\%$)
- Le code est révisé par au moins un autre membre de l'équipe
- La fonctionnalité est testée manuellement (interface ou cURL/Postman)
- Le code est commité et poussé sur GitHub (git push origin main)
- Le déploiement Render est validé (HTTP 200 sur l'endpoint)

5.9 Cérémonies Scrum réalisées

Cérémonie	Sprint 1	Sprint 2	Sprint 3	Sprint 4
Daily Scrum	✅ Done (#35)	✅ Done (#36)	✅ Done (#37)	✅ Done
Sprint Review	✅ Done (#38)	✅ Done (#39)	✅ Done (#40)	✅ Done
Sprint Retrospective	✅ Done (#41)	✅ Done (#42)	✅ Done (#43)	✅ Done

6. Difficultés Rencontrées et Leçons Apprises

6.1 Difficultés Techniques

Difficulté	Description	Solution apportée
Configuration Spring Security	La configuration des patterns d'accès (endpoints publics vs sécurisés) causait des HTTP 403 inattendus	Séparation explicite avec SecurityFilterChain, configuration CORS avec @CrossOrigin(*)
Intégration SendGrid	Migration de JavaMailSender SMTP vers SendGrid API v3 (SMTP bloqué par Render)	Utilisation de l'API REST SendGrid v3 avec RestTemplate, clé API en variable d'environnement
Performances BDD distante	Latence élevée vers Railway MySQL (connexions multiples = timeouts)	Configuration HikariCP connection pool (max 5 connexions, timeout 30s)
Déploiement Render Free Tier	Cold start de 30-50 secondes après inactivité (service s'endort)	UptimeRobot ping toutes les 5 minutes pour maintenir le service actif
Héritage JPA JOINED	Requêtes JPQL complexes avec héritage (Utilisateur → Etudiant/Bibliothécaire)	Utilisation de @DiscriminatorColumn et requêtes natives quand nécessaire

6.2 Difficultés Organisationnelles

Difficulté	Impact	Solution
Coordination à distance	Synchronisation des 7 membres difficile	Daily Scrum sur WhatsApp + GitHub pour le code
Conflits Git	Merge conflicts fréquents sur les mêmes fichiers	Convention de branches par feature (feature/emprunts, feature/auth)
Estimation des tâches	Sous-estimation de la complexité Spring Security	Révision du Sprint 1 en cours de sprint (ajout 2 jours)

6.3 Leçons Apprises

Comme leçons apprises, nous avons :

- La documentation technique (Javadoc, README) est essentielle dès le début du projet
- Les tests unitaires doivent être écrits en parallèle du code, pas à la fin
- Spring Security nécessite une configuration très précise — lire la documentation officielle
- Le déploiement cloud a ses contraintes (cold start, variables d'environnement, connexions BDD)
- La méthode Scrum structure efficacement le travail d'équipe et permet de livrer régulièrement
- GitHub Projects est un outil pratique pour suivre l'avancement et documenter les sprints

Conclusion

Le projet de Gestion de Bibliothèque Universitaire a été mené à bien en respectant l'ensemble des exigences du cahier des charges du Dr OUEDRAOGO Moïse. Les résultats obtenus sont les suivants :

- 10 exigences fonctionnelles (EF01 à EF10) entièrement implémentées et validées
- 6 exigences non fonctionnelles (ENF01 à ENF06) satisfaites
- 5 Design Patterns du Gang of Four implémentés (objectif : minimum 4)
- 93% de couverture de tests unitaires avec 61 tests JUnit 5 (objectif : $\geq 80\%$)
- 4 Sprints Scrum réalisés avec toutes les cérémonies (Daily Scrum, Sprint Review, Sprint Retrospective)
- 15 User Stories (US01 à US15) toutes en statut Done
- Application déployée et accessible publiquement sur infrastructure cloud

L'application est accessible publiquement à l'adresse :

[*https://bibliotheque-univ-nz1-5.onrender.com*](https://bibliotheque-univ-nz1-5.onrender.com)

Ce projet nous a permis de mettre en pratique les concepts fondamentaux du Génie Logiciel : conception UML, Design Patterns, architecture en couches, tests unitaires, déploiement cloud et méthodologie Agile Scrum. Ces compétences sont directement applicables dans un contexte professionnel de développement logiciel.

Annexes

A. URLs et Accès

Ressource	URL
Application web	https://bibliotheque-univ-nz1-5.onrender.com
Code source GitHub	https://github.com/moumounibanao0-afk/bibliotheque-univ-nz1
API Base URL	https://bibliotheque-univ-nz1-5.onrender.com/api
GitHub Projects — Product Backlog	https://github.com/users/moumounibanao0-afk/projects/12
GitHub Projects — Kanban Sprints	https://github.com/users/moumounibanao0-afk/projects/15

B. Comptes de Test

Rôle	Email	Mot de passe
Administrateur	admin@unz.bf	Admin123
Bibliothécaire	sali.traore@unz.bf	Sali123
Étudiant	fatima.kabore@unz.bf	Fatima123

C. Principaux Endpoints API

Méthode	Endpoint	Accès requis	Description
POST	/api/auth/login	Public	Authentification — retourne token JWT
POST	/api/utilisateurs/etudiant	Public	Inscription étudiant (INE)
GET	/api/ouvrages	Public	Liste des ouvrages du catalogue
GET	/api/categories	Public	Liste des catégories
POST	/api/ouvrages	Bibliothécaire+	Ajouter un ouvrage
PUT	/api/ouvrages/{id}	Bibliothécaire+	Modifier un ouvrage
DELETE	/api/ouvrages/{id}	Bibliothécaire+	Supprimer un ouvrage
POST	/api/emprunts	Authentifié	Créer un emprunt
PUT	/api/emprunts/{id}/retour	Bibliothécaire+	Retourner un ouvrage
GET	/api/emprunts/{id}/penalite	Bibliothécaire+	Calculer pénalité de retard
POST	/api/reservations	Authentifié	Créer une réservation
PUT	/api/reservations/{id}/annuler	Authentifié	Annuler une réservation
GET	/api/statistiques	Bibliothécaire+	Statistiques globales
GET	/api/statistiques/retards	Bibliothécaire+	Rapport des retards
GET	/api/utilisateurs	Administrateur	Liste de tous les utilisateurs
POST	/api/notifications/test	Bibliothécaire+	Envoyer une notification email test

D. Structure des Packages Java

Package	Contenu	Pattern
com.bibliotheque.model	Utilisateur, Etudiant, Bibliothecaire, Ouvrage, Categorie, Emprunt, Reservation, Role (enum), StatutEmprunt (enum), StatutReservation (enum)	Entités JPA
com.bibliotheque.repository	CategorieRepository, EmpruntRepository, OuvrageRepository, ReservationRepository, UtilisateurRepository	Pattern Repository
com.bibliotheque.service	EmpruntService, NotificationService, OuvrageService, ReservationService	Service Layer
com.bibliotheque.controller	AuthController, CategorieController, EmpruntController, HomeController, OuvrageController, ReservationController, StatistiqueController, UtilisateurController	Pattern Facade
com.bibliotheque.config	DatabaseConfig, JwtFilter, JwtUtil, SecurityConfig	Configuration + Factory
com.bibliotheque.strategy	CalculPenalite (interface), PenaliteStandard, PenaliteBoursier	Pattern Strategy
com.bibliotheque.observer	Observable (interface), Observateur (interface), OuvrageDisponible	Pattern Observer
com.bibliotheque.factory	UtilisateurFactory	Pattern Factory